

# EVALUACIÓN PA4 - PROGRAMACIÓN ORIENTADA A OBJETOS

|              |  |
|--------------|--|
| Institución: |  |
| Curso:       |  |
| Código:      |  |
| Docente(s):  |  |
| Sección:     |  |
| Fecha:       |  |

**Integrantes del equipo (4 integrantes, máximo 5):**

1. \_\_\_\_\_
2. \_\_\_\_\_
3. \_\_\_\_\_
4. \_\_\_\_\_
5. \_\_\_\_\_

## 1. Base de la evaluación en el sílabo y distribución temática

De acuerdo con el sílabo, la PA4 se aplica en la sesión 13. Como la PA3 se ubica en la sesión 10, esta evaluación cubre de manera lógica y progresiva los aprendizajes de las sesiones 11, 12 y 13: colección de objetos, búsquedas y ordenamientos, uso de HashSet y colecciones genéricas, herencia, clases abstractas, interfaces y sobrescritura de métodos.

## 2. Descripción breve para aula virtual

La PA4 evalúa la capacidad del equipo para diseñar e implementar una solución en Java que integre colecciones de objetos y una jerarquía de clases, aplicando herencia, clase abstracta, interfaz, sobrescritura de métodos, búsquedas y ordenamientos. La entrega se realizará mediante repositorio en GitHub. En el README del repositorio se deberá incluir el enlace al video de YouTube donde el equipo explique la solución desarrollada con cámaras encendidas.

## 3. Datos generales de la evaluación

- Modalidad: grupal colaborativa
- Integrantes por equipo: 4 estudiantes, máximo 5
- Duración sugerida: 120 minutos de desarrollo + entrega del repositorio y evidencias
- Entregable obligatorio: enlace del repositorio GitHub del equipo
- Evidencia adicional obligatoria: enlace de video en YouTube dentro del README del repositorio, con cámaras prendidas

## 4. Competencias o aprendizajes esperados

- Crear y manipular colecciones de objetos en Java para almacenar información de un caso aplicado.
- Implementar búsquedas, recorridos, unicidad de datos y ordenamientos básicos sobre colecciones.
- Diseñar una jerarquía de clases empleando herencia, clase abstracta, interfaz y sobrescritura de métodos.
- Sustentar decisiones técnicas del equipo de manera clara, ordenada y colaborativa.

- Organizar la entrega técnica en un repositorio GitHub con documentación mínima y evidencia audiovisual.

## 5. Examen

Caso propuesto: una tienda académica desea mejorar su sistema de control de productos y categorías. El equipo deberá desarrollar una solución en Java, basada en consola o con interfaz simple según el avance del curso, que gestione objetos de productos y demuestre el uso integrado de colecciones y jerarquía de clases.

### 5.1 Consigna general

Desarrolle en equipo un programa que permita registrar, almacenar y consultar productos de la tienda académica. La solución debe evidenciar el uso de una colección de objetos, una colección adicional para controlar la unicidad de un dato, y una jerarquía de clases que incluya al menos una clase abstracta, una interfaz y sobrescritura de métodos.

### 5.2 Requerimientos mínimos del desarrollo

- Definir una clase abstracta base con atributos comunes del dominio.
- Definir al menos dos clases derivadas que hereden de la clase abstracta y representen tipos distintos de producto.
- Definir una interfaz con al menos un comportamiento obligatorio y aplicarla en una o más clases del sistema.
- Crear y utilizar una colección de objetos genérica para almacenar los productos registrados.
- Utilizar una segunda colección para validar unicidad de información relevante, por ejemplo códigos de producto mediante HashSet.
- Implementar un menú o flujo de acciones que permita registrar, listar y consultar productos.
- Implementar al menos una búsqueda por código o nombre y un ordenamiento de la colección.
- Aplicar sobrescritura de métodos para mostrar diferencias entre los tipos de producto definidos.
- Presentar el trabajo final en un repositorio GitHub del equipo.
- Agregar en el README del repositorio el enlace del video de YouTube con la explicación del trabajo resuelto y cámaras prendidas.

### 5.3 Actividades específicas

1. Analizar el caso y acordar la estructura general de clases del sistema.
2. Modelar la jerarquía de clases y justificar la elección de la clase abstracta y la interfaz.
3. Programar el registro de objetos en una colección genérica.
4. Aplicar una estructura de unicidad con HashSet u otra colección apropiada del temario.
5. Implementar una opción de búsqueda y una opción de ordenamiento sobre la colección de objetos.
6. Ejecutar pruebas básicas del sistema y registrar evidencias en el repositorio.
7. Documentar en el README: objetivo, integrantes, instrucciones de ejecución, decisiones tomadas y enlace del video de YouTube.

### 5.4 Indicaciones claras para los estudiantes

- La evaluación se desarrolla en equipos de 4 estudiantes, con un máximo de 5.
- El trabajo debe ser original del equipo y coherente con los temas abordados en el curso hasta la sesión 13.
- No se evaluará cantidad de código, sino pertinencia técnica, claridad del diseño y funcionamiento.
- La entrega debe realizarse mediante un repositorio GitHub compartido por el equipo.
- El README del repositorio debe contener: nombre de la evaluación, integrantes, breve explicación de la solución, instrucciones de ejecución y enlace del video de YouTube.
- El video de YouTube debe mostrar a los integrantes con cámaras prendidas y explicar: cómo resolvieron la evaluación, por qué llegaron a esa solución y qué decisiones tomaron durante el desarrollo.

- El enlace del repositorio y el enlace del video deben estar operativos al momento de la revisión.
- Se recomienda realizar commits que evidencien trabajo colaborativo y organización del desarrollo.

## 6. Rúbrica de calificación (20 puntos)

| Criterio                                                 | Excelente                                                                                                                    | Logrado                                                                                | En proceso                                                           | Inicial                                                        | Puntaje máx. |
|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------|----------------------------------------------------------------|--------------|
| Diseño de jerarquía de clases (4 pts)                    | Clase abstracta, derivadas e interfaz coherentes; sobrescritura correcta. (4)                                                | La jerarquía funciona con leves inconsistencias. (3)                                   | La jerarquía es parcial o poco coherente. (2)                        | No se evidencia diseño suficiente. (0-1)                       | 4            |
| Uso de colecciones de objetos y unicidad (4 pts)         | Colección genérica y estructura de unicidad bien implementadas. (4)                                                          | Implementación funcional con detalles menores. (3)                                     | Implementación parcial o con errores frecuentes. (2)                 | No implementa adecuadamente las colecciones del temario. (0-1) | 4            |
| Búsqueda, ordenamiento y funcionamiento (4 pts)          | Búsqueda y ordenamiento correctos; funcionamiento claro. (4)                                                                 | Funciona con detalles menores por mejorar. (3)                                         | Solo una operación funciona adecuadamente o hay fallas notorias. (2) | No evidencia funcionamiento suficiente. (0-1)                  | 4            |
| Pertinencia técnica y resolución del caso (3 pts)        | Responde al caso con lógica consistente y decisiones sustentadas. (3)                                                        | Responde al caso con algunas omisiones menores. (2)                                    | La solución es parcial. (1)                                          | No responde de forma pertinente. (0)                           | 3            |
| Repositorio GitHub y README (3 pts)                      | Repositorio ordenado y funcional; README completo. (3)                                                                       | Repositorio funcional; README suficiente. (2)                                          | Entrega incompleta o README parcial. (1)                             | No entrega repositorio funcional o README. (0)                 | 3            |
| Video de YouTube en README con cámaras prendidas (2 pts) | Enlace operativo; explican cómo resolvieron, por qué eligieron la solución y qué decisiones tomaron; cámaras encendidas. (2) | Video enlazado, pero la explicación es parcial o alguna condición es insuficiente. (1) | No cumple la evidencia audiovisual solicitada. (0)                   | No cumple la evidencia audiovisual solicitada. (0)             | 2            |

**Puntaje total:** \_\_\_\_ / 20